

Dept. of Electronics and Electrical Communication Engineering

Indian Institute of Technology, Kharagpur

VLSI DESIGN LAB [EC69216]



EXPERIMENT - 4

Square Root Module in Verilog

Group No. 4

- **Member 1:** Arup Basak 22EC37021
- **Member 2:** Chiradip Biswas 22EC37004

1 Aim of the Experiment

Implement a design with iterative Babylonian method to compute the square root of an 8-bit integer, yielding a 4-bit integer and 4-bit fractional fixed-point result. The architecture is partitioned into a finite state machine (FSM) control unit, a computational datapath, and a custom fixed-point divider.

1. perform behavioural simulation
2. perform post implementation simulation and utilization reports, timing summaries.

2 Introduction and Top-Level Architecture

The top-level module (`sqrt_top`) serves as the structural wrapper that connects the Control Unit and the Datapath. It accepts an 8-bit integer input N (e.g., 255) and an active-high `start` signal. The output is an 8-bit fixed-point number formatted in Q4.4 (4 bits integer, 4 bits fraction) alongside a `done` flag.

The system implements the Babylonian iteration:

$$Guess_{new} = \frac{Guess + \frac{N}{Guess}}{2}$$

This loop continues until the difference between the $Guess$ and the intermediate quotient ($\frac{N}{Guess}$) is negligible.

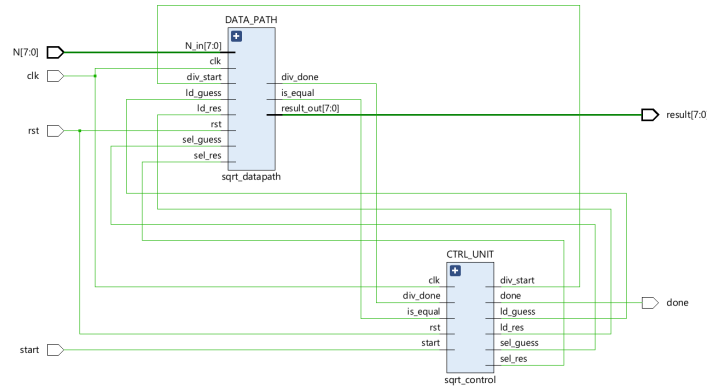


Figure 1: The top module schematic

- **Overflow Protection (Safe Load):** A critical safety multiplexer prevents the Result register from loading quotient values ≥ 16.0 . It checks the 9th bit (`res_next_val[8]`); if it is high, the load is blocked, preventing overflow in the final 8-bit output.

5 Fixed-Point Divider (`fixed_divider`)

The fixed-point divider is the most computationally complex module. It computes the operation $Quotient = \frac{Dividend}{Divisor}$ using an iterative shift-and-subtract algorithm. The inputs are an 8-bit integer dividend and a 12-bit Q8.4 divisor, producing a 9-bit Q5.4 quotient.

5.1 Scaling by 256 (Fractional Precision)

To generate a quotient with 4 bits of fractional precision (Q.4), the mathematics require the dividend to be multiplied by 2^4 . However, because the divisor is already scaled by 2^4 (Q8.4 format), the dividend must actually be multiplied by 2^8 (scaled by 256) to maintain the correct fixed-point alignment.

In hardware, this multiplication is achieved without logic gates by using a 21-bit shift register. The 8-bit dividend is placed at the bottom of a 13-bit comparison window, followed by 8 trailing zeros (the shift buffer): `{5'b0, dividend, 8'b0}`. This physical placement mathematically scales the dividend by 256 prior to the start of division.

5.2 The Shift-and-Subtract Algorithm

The division operates over a 3-state FSM using a 21-bit `remainder_reg`.

- **State 0 (IDLE & MSB Optimization):** When triggered, the module aligns the dividend as described above. To save a clock cycle, it immediately compares the top 13 bits of the remainder register against the 12-bit divisor. If the top 13 bits are greater than or equal to the divisor, it subtracts the divisor and sets the MSB of the 9-bit quotient to 1. It then initializes a counter to 8 and transitions to the loop.
- **State 1 (CALC):** This loop executes 8 times to find the remaining bits. In each cycle:
 1. The 21-bit remainder is shifted left by 1 bit, bringing the next value into the 13-bit comparison window.
 2. The top 13 bits are compared to the divisor.
 3. If greater or equal, the divisor is subtracted from the top bits, and a '1' is shifted into the LSB of the quotient register.
 4. If less, no subtraction occurs, and a '0' is shifted into the quotient register.

The counter decrements until it reaches 1, after which the FSM transitions to finish.

- **State 2 (FINISH):** The `done` flag is asserted, signaling the main Control Unit that the division is complete and the quotient is ready.

6 Observations

To interpret the results, in the plots, input is shown in decimal (8bit, so between 1 to 255), and the result is shown in 8bit hex (upper nibble for integer part lower for fraction).

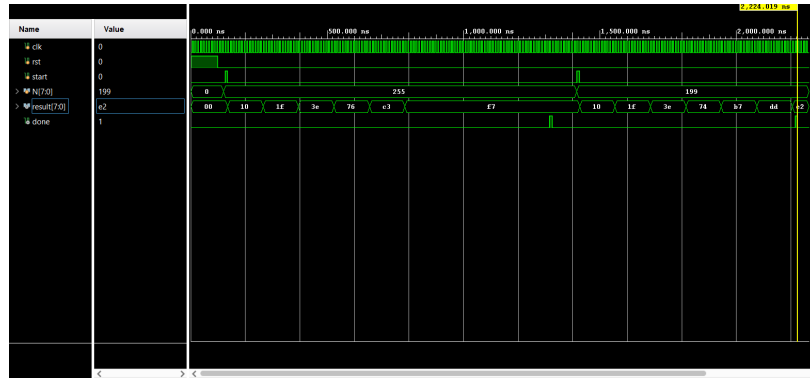


Figure 3: Behavioural timing simulation

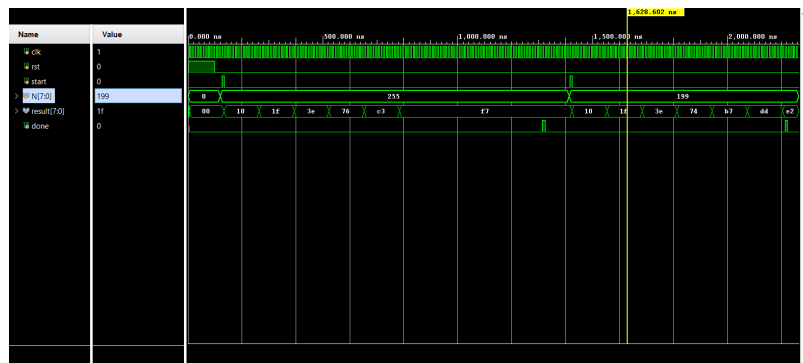


Figure 4: post implementation timing simulation

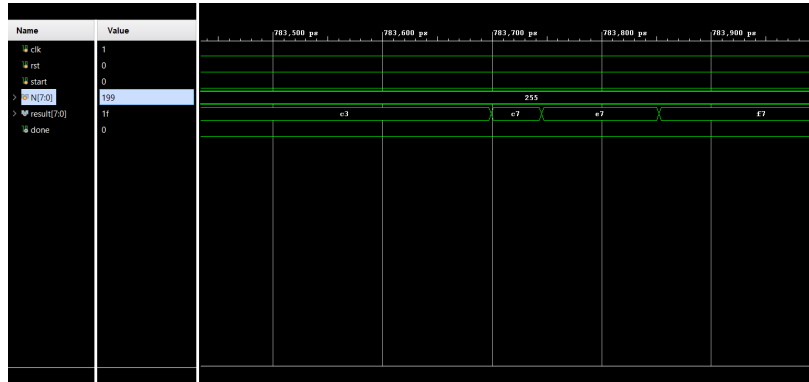


Figure 5: Post implementation simulation spurious states

Name	^1	Slice LUTs (41000)	Slice Registers (82000)	Slice (10250)	LUT as Logic (41000)	Bonded IOB (285)	BUFGCTRL (32)
sqrt_top		103	76	33	103	20	1
CTRL_UNIT (sqrt_control)		5	6	4	5	0	0
DATA_PATH (sqrt_datapath)		98	70	33	98	0	0
reg_guess (register_12bit)		50	12	16	50	0	0
reg_res (register_9bit)		9	17	7	9	0	0
u_comp (comparator_equal)		4	0	6	4	0	0
u_divider (fixed_divider)		44	41	21	44	0	0

Figure 6: post implementation utilization report

Name	Slack	^1	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Req
Path 1	∞		9	3	23	N[3]	DATA_PATH/u_div_reg_reg[18]/D	5.049	2.069	2.980	
Path 2	∞		9	3	23	N[3]	DATA_PATH/u_div_reg_reg[16]/D	4.935	2.078	2.857	
Path 3	∞		8	3	23	N[3]	DATA_PATH/u_div_reg_reg[12]/D	4.875	2.018	2.857	
Path 4	∞		9	3	23	N[3]	DATA_PATH/u_div_reg_reg[19]/D	4.806	2.103	2.704	
Path 5	∞		8	3	23	N[3]	DATA_PATH/u_div_reg_reg[13]/D	4.796	2.096	2.700	
Path 6	∞		9	4	11	DATA_PATH/reg_q_out_reg[8]/C	CTRL_UNIT/FSM_o_state_reg[2]/D	4.794	1.444	3.350	
Path 7	∞		9	4	11	DATA_PATH/reg_q_out_reg[8]/C	CTRL_UNIT/FSM_o_state_reg[3]/D	4.790	1.444	3.346	
Path 8	∞		7	3	23	N[3]	DATA_PATH/u_div_reg_reg[11]/D	4.775	1.941	2.834	
Path 9	∞		8	3	23	N[3]	DATA_PATH/u_div_reg_reg[15]/D	4.760	2.057	2.704	
Path 10	∞		9	3	23	N[3]	DATA_PATH/u_div_reg_reg[17]/D	4.672	2.147	2.525	

Figure 7: Timing summary of the post implementation

6.1 Top Module Code

```
1  `timescale 1ns / 1ps
2
3  module sqrt_top (
4      input wire clk,
5      input wire rst,
6      input wire start,
7      input wire [7:0] N,          // Input Integer (e.g. 255)
8      output wire [7:0] result,    // Output Fixed Point (e.g. 15.9 -> 1111.1110)
9      output wire done
10 );
11
12 // =====
13 // INTERCONNECT WIRES
14 // =====
15
16 // Control Signals (Control -> Datapath)
17 wire ld_guess;
18 wire sel_guess;
19 wire ld_res;
20 wire sel_res;
21 wire div_start;
22
23 // Status Signals (Datapath -> Control)
24 wire is_equal;
25 wire div_done;
26
27 // =====
28 // INSTANTIATION
29 // =====
30
31 // 1. Control Unit Instance
32 sqrt_control CTRL_UNIT (
33     .clk(clk),
34     .rst(rst),
35     .start(start),
36
37     // Inputs from Datapath
38     .is_equal(is_equal),
39     .div_done(div_done),
40
41     // Outputs to Datapath
42     .ld_guess(ld_guess),
43     .sel_guess(sel_guess),
44     .ld_res(ld_res),
45     .sel_res(sel_res),
46     .div_start(div_start),
47
48     // System Output
49     .done(done)
50 );
51
52 // 2. Datapath Instance
53 sqrt_datapath DATA_PATH (
54     .clk(clk),
55     .rst(rst),
56
57     // Data Input
58     .N_in(N),
59
60     // Control Inputs
61     .ld_guess(ld_guess),
62     .sel_guess(sel_guess),
63     .ld_res(ld_res),
64     .sel_res(sel_res),
65     .div_start(div_start),
66
67     // Status Outputs
68     .is_equal(is_equal),
69     .div_done(div_done),
70
```

```

71         // Data Output
72         .result_out(result)
73     );
74
75 endmodule

```

Listing 1: sqrt_top.v

6.2 Control Unit (control_unit.v)

The finite state machine governing the algorithm iterations[cite: 5].

```

1  `timescale 1ns / 1ps
2
3  module sqrt_control (
4      input wire clk, rst, start,
5
6      // Status Inputs from Datapath
7      input wire is_equal,
8      input wire div_done,
9
10     // Control Outputs to Datapath
11     output reg ld_guess,
12     output reg sel_guess,
13     output reg ld_res,
14     output reg sel_res,
15     output reg div_start,
16
17     // System Output
18     output reg done
19 );
20
21     // State Encoding
22     localparam S_IDLE      = 3'd0;
23     localparam S_CHECK     = 3'd1;
24     localparam S_CALC_AVG  = 3'd2;
25     localparam S_DIV_START = 3'd3;
26     localparam S_DIV_WAIT  = 3'd4;
27     localparam S_DONE      = 3'd5;
28
29     reg [2:0] state, next_state;
30
31     // =====
32     // PROCESS 1: State Register (Sequential Logic)
33     // Uses Non-Blocking (<=) to synchronize with clock.
34     // =====
35     always @(posedge clk or posedge rst) begin
36         if (rst) state <= S_IDLE;
37         else     state <= next_state;
38     end
39
40     // =====
41     // PROCESS 2: Next State Logic (Combinational Logic)
42     // Uses Blocking (=) for immediate logic calculation.
43     // =====
44     always @(*) begin
45         // Default: Stay in current state (Safety)
46         next_state <= state;
47
48         case (state)
49             S_IDLE: begin
50                 if (start) next_state <= S_CHECK;
51                 else       next_state <= S_IDLE;
52             end
53
54             S_CHECK: begin
55                 if (is_equal) next_state <= S_DONE;
56                 else          next_state <= S_CALC_AVG;
57             end

```

```

58
59     S_CALC_AVG: begin
60         next_state <= S_DIV_START;
61     end
62
63     S_DIV_START: begin
64         next_state <= S_DIV_WAIT;
65     end
66
67     S_DIV_WAIT: begin
68         if (div_done) next_state <= S_CHECK;
69         else          next_state <= S_DIV_WAIT;
70     end
71
72     S_DONE: begin
73         // Handshake: Wait for user to release start to go back to IDLE
74         // This prevents restarting immediately if start is held high
75         if (start == 0) next_state <= S_IDLE;
76         else            next_state <= S_DONE;
77     end
78
79     default: next_state <= S_IDLE;
80 endcase
81 end
82
83 // =====
84 // PROCESS 3: Output Logic (Combinational Logic)
85 // Uses Blocking (=) for immediate signal generation (Mealy/Moore).
86 // =====
87 always @(*) begin
88     // 1. DEFAULT VALUES (CRITICAL to prevent latches/races)
89     // If you don't assign a value in a case, it defaults to 0.
90     ld_guess  <= 0;
91     sel_guess <= 0;
92     ld_res    <= 0;
93     sel_res   <= 0;
94     div_start <= 0;
95     done      <= 0;
96
97     case (state)
98     S_IDLE: begin
99         if (start) begin
100             // Initialize Registers (Mealy Output)
101             ld_guess  <= 1;
102             sel_guess <= 0; // Load N<4
103             ld_res    <= 1;
104             sel_res   <= 0; // Load 1.0
105         end
106     end
107
108     // S_CHECK: No outputs active (defaults to 0)
109
110     S_CALC_AVG: begin
111         // Update Guess with Average
112         ld_guess  <= 1;
113         sel_guess <= 1; // Load (Guess+Res)/2
114     end
115
116     S_DIV_START: begin
117         // Pulse Start for Divider
118         div_start <= 1;
119     end
120
121     S_DIV_WAIT: begin
122         if (div_done) begin
123             // Update Res with Divider Output (Mealy Output)
124             // Note: The Datapath handles the "Safe Load" logic check
125             ld_res  <= 1;
126             sel_res <= 1;
127         end
128     end
129

```

```

130         S_DONE: begin
131             done <= 1;
132         end
133     endcase
134 end
135
136 endmodule

```

Listing 2: sqrt_control.v

6.3 Datapath (datapath.v)

The data computation unit and sub-modules for the algorithm[cite: 6].

```

1  `timescale 1ns / 1ps
2
3  // ... (Keep your helper modules register_12bit, mux_2to1, etc. as they are) ...
4  // =====
5  // HELPER MODULES
6  // =====
7
8  // 1. 12-bit Register (For Guess)
9  module register_12bit (
10     input wire clk, rst, load,
11     input wire [11:0] d_in,
12     output reg [11:0] q_out
13 );
14     always @(posedge clk or posedge rst) begin
15         if (rst) q_out <= 0;
16         else if (load) q_out <= d_in;
17     end
18 endmodule
19
20 // 2. 9-bit Register (For Result)
21 module register_9bit (
22     input wire clk, rst, load,
23     input wire [8:0] d_in,
24     output reg [8:0] q_out
25 );
26     always @(posedge clk or posedge rst) begin
27         if (rst) q_out <= 0;
28         else if (load) q_out <= d_in;
29     end
30 endmodule
31
32 // 3. MUX 2-to-1 (12-bit)
33 module mux_2to1_12bit (
34     input wire sel,
35     input wire [11:0] in0,
36     input wire [11:0] in1,
37     output wire [11:0] out
38 );
39     assign out = (sel) ? in1 : in0;
40 endmodule
41
42 // 4. MUX 2-to-1 (9-bit)
43 module mux_2to1_9bit (
44     input wire sel,
45     input wire [8:0] in0,
46     input wire [8:0] in1,
47     output wire [8:0] out
48 );
49     assign out = (sel) ? in1 : in0;
50 endmodule
51
52 // 5. Averager: (Guess + Res) / 2
53 // Structurally adds 12-bit Guess and 9-bit Res (Zero padded), then shifts.
54 module adder_avg (
55     input wire [11:0] guess_in,

```

```

56     input wire [8:0] res_in,
57     output wire [11:0] avg_out
58 );
59 wire [12:0] sum;
60 // Guess (12bit) + Res (9bit padded to 12)
61 assign sum = guess_in + {3'b0, res_in};
62 // Divide by 2 (take bits [12:1])
63 assign avg_out = sum[12:1];
64 endmodule
65
66 // 6. Comparator (Guess == Res) (approx)
67 module comparator_equal (
68     input wire [11:0] guess,
69     input wire [8:0] res,
70     output reg is_equal
71 );
72 wire [11:0] res_extended;
73 wire [11:0] diff;
74
75 assign res_extended = {3'b0, res}; // Align Q5.4 to Q8.4
76
77 // Calculate absolute difference structurally
78 assign diff = (guess > res_extended) ? (guess - res_extended) : (res_extended - guess);
79
80 always @(*) begin
81     // Converge if values are equal OR difference is just 1 LSB
82     if (diff <= 12'd1) //<--adjust it
83         is_equal = 1;
84     else
85         is_equal = 0;
86 end
87 endmodule
88 module sqrt_datapath (
89     input wire clk, rst,
90     input wire [7:0] N_in,
91
92     // Control Signals
93     input wire ld_guess,
94     input wire sel_guess,
95     input wire ld_res, // Signal from Control Unit to load Res
96     input wire sel_res,
97     input wire div_start,
98
99     // Status Outputs
100    output wire is_equal,
101    output wire div_done,
102
103    // Final Data Output
104    output wire [7:0] result_out
105 );
106
107 // Internal Wires
108 wire [11:0] guess_curr;
109 wire [11:0] guess_next_val;
110 wire [11:0] guess_avg_calc;
111 wire [11:0] guess_init_val;
112
113 wire [8:0] res_curr;
114 wire [8:0] res_next_val;
115 wire [8:0] res_div_out;
116 wire [8:0] res_init_val;
117
118 // Internal Control Wire
119 wire ld_res_safe;
120
121 // -----
122 // 1. GUESS REGISTER LOGIC
123 // -----
124 assign guess_init_val = {N_in, 4'b0};
125
126 mux_2to1_12bit mux_guess (
127     .sel(sel_guess),

```

```

128         .in0(guess_init_val),
129         .in1(guess_avg_calc),
130         .out(guess_next_val)
131     );
132
133     register_12bit reg_guess (
134         .clk(clk),
135         .rst(rst),
136         .load(ld_guess),
137         .d_in(guess_next_val),
138         .q_out(guess_curr)
139     );
140
141     // -----
142     // 2. RESULT REGISTER LOGIC (WITH FIX)
143     // -----
144     assign res_init_val = 9'b000010000;
145
146     mux_2to1_9bit mux_res (
147         .sel(sel_res),
148         .in0(res_init_val),
149         .in1(res_div_out),
150         .out(res_next_val)
151     );
152
153     // --- THE FIX IS HERE ---
154     // If we are trying to load the Divider Output (sel_res=1),
155     // AND that output is >= 16.0 (bit 8 is 1, i.e., 1_0000_0000),
156     // THEN BLOCK THE LOAD.
157     //
158     // We only allow loading if:
159     // 1. It's initialization (sel_res=0)
160     // 2. OR It's a valid value (res_next_val[8] == 0)
161
162     assign ld_res_safe = ld_res & (~sel_res | ~res_next_val[8]);
163
164     register_9bit reg_res (
165         .clk(clk),
166         .rst(rst),
167         .load(ld_res_safe), // Use the safe signal
168         .d_in(res_next_val),
169         .q_out(res_curr)
170     );
171
172     // -----
173     // 3. ARITHMETIC LOGIC
174     // -----
175     adder_avg u_avg (
176         .guess_in(guess_curr),
177         .res_in(res_curr),
178         .avg_out(guess_avg_calc)
179     );
180
181     comparator_equal u_comp (
182         .guess(guess_curr),
183         .res(res_curr),
184         .is_equal(is_equal)
185     );
186
187     fixed_divider u_divider (
188         .clk(clk),
189         .rst(rst),
190         .start(div_start),
191         .dividend(N_in),
192         .divisor(guess_curr),
193         .quotient(res_div_out),
194         .done(div_done)
195     );
196
197     // -----
198     // 4. OUTPUT MAPPING
199     // -----

```

```

200     assign result_out = res_curr[7:0];
201
202 endmodule

```

Listing 3: sqrt_datapath.v

6.4 Fixed-Point Divider (fixed_divider.v)

The iterative shift-and-subtract hardware divider[cite: 7].

```

1  `timescale 1ns / 1ps
2
3  module fixed_divider (
4      input wire  clk, rst, start,
5      input wire  [7:0] dividend,    // 8-bit Integer (0-255)
6      input wire  [11:0] divisor,    // 12-bit Q8.4 Fixed Point
7      output reg  [8:0] quotient,    // 9-bit Result (Q5.4)
8      output reg  done
9  );
10
11     // Register width calculation:
12     // Window (13 bits) + Shift Buffer (8 bits) = 21 bits
13     // Window needs to be 1 bit wider than divisor (12 bits) for sign/overflow safety.
14     reg [20:0] remainder_reg;
15     reg [11:0] divisor_copy;
16     reg [3:0]  count;
17     reg [1:0]  state;
18
19     // Combinatorial signals for calculations
20     reg [20:0] remainder_calc;
21
22     always @(posedge clk or posedge rst) begin
23         if (rst) begin
24             state <= 0;
25             done <= 0;
26             quotient <= 0;
27             remainder_reg <= 0;
28             divisor_copy <= 0;
29             count <= 0;
30         end else begin
31             case (state)
32                 0: begin // IDLE
33                     done <= 0;
34                     if (start) begin
35                         divisor_copy <= divisor;
36
37                         // --- OPTIMIZATION: Process MSB (Bit 8) Immediately ---
38
39                         // 1. Align Dividend (Shift 8 effectively)
40                         // Window is top 13 bits [20:8].
41                         // We place 'dividend' such that it sits at the bottom of this window.
42                         // {5'b0 (pad), dividend (8), 8'b0 (shift buffer)}
43                         remainder_calc = {5'b0, dividend, 8'b0};
44
45                         // 2. Immediate Compare
46                         // Compare Top 13 bits vs Divisor (12 bits)
47                         if (remainder_calc[20:8] >= divisor) begin
48                             remainder_reg <= remainder_calc - {1'b0, divisor, 8'b0};
49                             quotient <= 9'b1; // Set MSB
50                         end else begin
51                             remainder_reg <= remainder_calc;
52                             quotient <= 9'b0; // Clear MSB
53                         end
54
55                         // 3. Setup Loop
56                         count <= 8;
57                         state <= 1;
58                     end
59                 end

```

```

60
61      1: begin // CALC
62          // 1. Shift Remainder Left by 1
63          remainder_calc = remainder_reg << 1;
64
65          // 2. Compare Top 13 bits vs Divisor
66          if (remainder_calc[20:8] >= divisor_copy) begin
67              // Subtract and shift in '1'
68              remainder_reg <= remainder_calc - {1'b0, divisor_copy, 8'b0};
69              quotient <= {quotient[7:0], 1'b1};
70          end else begin
71              // Just update remainder and shift in '0'
72              remainder_reg <= remainder_calc;
73              quotient <= {quotient[7:0], 1'b0};
74          end
75
76          // 3. Loop Control
77          count <= count - 1;
78          if (count == 1) state <= 2;
79      end
80
81      2: begin // FINISH
82          done <= 1;
83          state <= 0;
84      end
85  endcase
86  end
87  end
88  endmodule

```

Listing 4: fixed_divider.v

7 Discussion Chiradip Biswas 22EC37004

7.1 Register Widths and Initial Conditions (Limiting Cases)

The architecture must carefully manage fixed-point boundaries to prevent overflow while maintaining the required precision. The algorithm initializes with $Guess = N$ and $Result = 1.0$.

Given that the maximum input integer N is 255 (an 8-bit value), the system must be sized to accommodate this limiting case:

- **The Guess Register (12-bit Q8.4):** To initialize $Guess = 255$ with 4 bits of fractional precision, the integer 255 is padded with 4 zeros ($\{N_in, 4'b0\}$). This requires 8 integer bits and 4 fractional bits, necessitating a 12-bit register.
- **The Result Register (9-bit Q5.4):** While the final output is only an 8-bit Q4.4 number (max value 15.9375), the internal Result register is 9 bits. The 9th bit acts as an overflow safety mechanism. Because intermediate quotients during the first few iterations could potentially exceed 15.9375, a “Safe Load” logic block monitors the 9th bit (`res_next_val[8]`). If this bit is ‘1’ (indicating a value ≥ 16.0), the datapath blocks the load to prevent invalid data from corrupting the iteration.

7.2 Control Unit Algorithm

The system relies on a central Finite State Machine (FSM) to sequence the datapath[cite: 2, 3, 4]. The control algorithm follows a strict sequence:

1. **S_IDLE:** Waits for the `start` signal. It simultaneously loads the initial values into the Guess and Result registers.
2. **S_CHECK:** Evaluates the `is_equal` convergence flag. If true, the algorithm terminates; if false, it proceeds to update the Guess.
3. **S_CALC_AVG:** Asserts the control signals to load the computed average into the Guess register.
4. **S_DIV_START & S_DIV_WAIT:** Pulses the divider to begin operation. The FSM then pauses in a wait state until the divider asserts `div_done`, at which point it loads the new quotient into the Result register and loops back to the check state.
5. **S_DONE:** Signals completion. It requires the `start` signal to be de-asserted before returning to IDLE, acting as a structural handshake to prevent continuous looping.

7.3 The Averaging Module

The Babylonian method requires calculating $Guess_{new} = \frac{Guess + Result}{2}$ in each iteration. Floating-point division by 2 is computationally expensive, but fixed-point hardware simplifies this drastically.

The `adder_avg` module structurally combines these operations. It zero-pads the 9-bit Result to align it with the 12-bit Guess, computes a 13-bit sum (`guess_in + {3'b0, res_in}`), and achieves the division by 2 via a hardwired right-shift, taking bits [12:1] of the sum. This takes zero clock cycles to compute, resolving purely through combinational logic.

7.4 The Fixed-Point Divider Algorithm

The custom `fixed_divider` module is the core computational unit. Because the hardware operates without floating-point units, it relies on an iterative shift-and-subtract long division algorithm.

7.4.1 Scaling by 256 (2^8)

To generate a quotient with 4 bits of fractional precision (Q5.4) by dividing an integer by a Q8.4 divisor, the dividend must be scaled. Mathematically, to retain 4 fractional bits, the integer N must be multiplied by 2^4 . However, because the divisor is *also* scaled by 2^4 (Q8.4), N must be multiplied by 2^8 (scaled by 256) to ensure the fixed-point decimal aligns correctly after division.

In hardware, this is accomplished by using a 21-bit remainder register. The 8-bit dividend is loaded into bits [15:8], effectively padding it with 8 trailing zeros (`{5'b0, dividend, 8'b0}`). This physical placement mathematically scales the input by 256 instantly.

7.4.2 Divider State Diagram

The multi-cycle division is governed by a 3-state FSM:

- **State 0 (IDLE / MSB Optimization):** The divider waits for a start pulse. To optimize performance, it immediately compares the top 13 bits of the aligned dividend against the divisor. It subtracts if possible, calculates the Most Significant Bit (MSB) of the quotient in cycle zero, sets the loop counter to 8, and transitions to State 1.

- **State 1 (CALC):** This state represents the iterative loop. Over 8 clock cycles, the 21-bit remainder is shifted left by 1. The top 13 bits are compared to the divisor. If $\geq$, the divisor is subtracted and a '1' is shifted into the quotient. If $<$, no subtraction occurs and a '0' is shifted in. Once the counter hits 1, it transitions to State 2.
- **State 2 (FINISH):** The **done** flag is raised, alerting the main control unit that the new Result is ready, and the FSM resets to State 0.

8 Discussion Arup Basak 22EC37021

- The transistor sizing was systematically derived using standard square-law equations for the saturation region, utilizing the characterized process parameters ($K'_n = 183.8 \mu\text{A}/\text{V}^2$, $K'_p = 46.8 \mu\text{A}/\text{V}^2$, and $V_{th} = 0.5\text{V}$). By judiciously partitioning the voltage margins derived from the given Input Common Mode Range (ICMR), the target quiescent currents of $200 \mu\text{A}$ and $800 \mu\text{A}$ for the first and second stages were successfully established.
- The simulation yielded an open-loop flat band gain of 48.6 dB. This is consistent with theoretical expectations for a two-stage topology, where the differential input stage provides high voltage gain and the common-source second stage provides additional gain along with the necessary current drive for the 1 pF load.
- The necessity of frequency compensation was starkly highlighted by the uncompensated phase margin, which was measured at a highly unstable 2.6° . The implementation of Miller compensation (using a capacitor and a series nulling resistor) effectively performed pole-splitting, pulling the dominant pole inward and pushing the non-dominant pole outward. This drastically improved the phase margin to 57.01° , satisfyingly close to the 60° design target.
- A clear trade-off between stability and bandwidth was observed during compensation. While the phase margin was corrected, the open-loop 3-dB bandwidth dropped significantly from 80 MHz in the uncompensated circuit to 643 kHz in the compensated circuit, directly demonstrating the effect of the lowered dominant pole frequency.
- When configured as a closed-loop unity-gain buffer, the amplifier exhibited an impressive extended 3-dB bandwidth of 290 MHz. The output linearity range spanned from 0 V to 1.567 V, validating the calculated output swing specifications and confirming that the amplifier correctly tracks the input signal without severe clipping or distortion.
- The circuit achieved a high positive slew rate of 410.88 MV/sec during large-signal transient operation. This robust transient response indicates that the $200 \mu\text{A}$ tail current in the differential stage is more than sufficient to rapidly charge and discharge the internal compensation capacitor, while the $800 \mu\text{A}$ second-stage current adequately drives the 1 pF external load.
- The total DC power dissipation was recorded at 2.1 mW. Considering the total designated quiescent current of 1 mA across both stages, this power consumption accurately reflects the expected static power draw, confirming that the DC operating points of the transistors remained well within the required saturation regions.